

Aplikacje Mobilne: Nawigacja Rowerowa

Stanisław Fiedler 160250

Michał Łatka 160263

1. Architektura aplikacji

Aplikacja została zaprojektowana w architekturze klient-serwer.

- klient: aplikacja Flutter (Dart)
- serwer routingowy: OSRM (Open Source Routing Machine)
- serwer danych i autoryzacji: PocketBase

Komunikacja pomiędzy warstwami odbywa się poprzez zapytania HTTP.

2. Funkcjonalności

2.1. Baza danych zdalna

Zdalna baza danych obsługiwana jest przez PocketBase uruchomiony jako docker na serwerze. Reguły dostępu skonfigurowane w PocketBase uniemożliwiają odczyt lub modyfikację danych należących do innych użytkowników

```
final pbProvider = Provider<PocketBase>((ref) {  
  final prefs = ref.watch(sharedPreferencesProvider);  
  const storageKey = 'pb_auth';  
  final cachedAuth = prefs.getString(storageKey);  
  
  final authStore = AsyncAuthStore(  
    initial: cachedAuth,  
    save: (String data) async {  
      await prefs.setString(storageKey, data);  
    },  
    clear: () async {  
      await prefs.remove(storageKey);  
    },  
  );  
  
  return PocketBase('http://192.168.1.102:8090', authStore: authStore);  
});  
  
Future<List<String>> _fetchFromDatabase(String userId) async {  
  final pb = ref.read(pbProvider);  
  try {  
    final records = await pb.collection('search_history').getList(  
      filter: 'user = "$userId"',  
      sort: '-created',  
    );  
    return records.items.map((item) => item.getStringValue('query')).toList();  
  } catch (_) {  
    return [];  
  }  
}
```

2.2. Interfejs użytkownika w całości oparty na Material Design

Interfejs aplikacji został wykonany w technologii Flutter z wykorzystaniem Material Design 3. Fragment widgetu:

```

return Scaffold(
  appBar: AppBar(title: const Text('Route Result')),
  floatingActionButton: routes.isNotEmpty
    ? FloatingActionButton.extended(
        onPressed: () {
          Navigator.of(context).push(
            MaterialPageRoute(
              builder: (context) => ActiveNavigationScreen(
                selectedRoute: routes[selectedRouteIndex],
              ),
            ),
          );
        },
        label: const Text('Start Route'),
        icon: const Icon(Icons.navigation),
        backgroundColor: Colors.blue,
      )
    : null,
  body: SafeArea(
    child: LayoutBuilder(
      builder: (BuildContext cntx, BoxConstraints constraints) {
        return Column(
          children: [
            LimitedBox(
              maxHeight: constraints.maxHeight / 3,
              child: ListView.builder(
                shrinkWrap: true,
                itemCount: widget.waypoints.length,
                itemBuilder: (BuildContext cntx, int index) {
                  return getLabel(index);
                },
              ),
            ),
          ],
        ),
      ),
    ),
  ),
);

```

2.3. Zastosowanie bezpiecznych mechanizmów identyfikacji i uwierzytelniania

Jako system autoryzacji wykorzystano PocketBase. Każdy użytkownik posiada własny zestaw zapisanych adresów. Reguły dostępu skonfigurowane w PocketBase uniemożliwiają odczyt lub modyfikację danych należących do innych użytkowników

```

class AuthNotifier extends Notifier<AuthState> {
  @override
  AuthState build() {
    final pb = ref.watch(pbProvider);
    return AuthState(isAuthenticated: pb.authStore.isValid);
  }

  Future<void> login(String email, String password) async {
    state = AuthState(isLoading: true);
    final pb = ref.read(pbProvider);

    try {
      await pb.collection('users').authWithPassword(email, password);
      state = AuthState(isAuthenticated: true);
    } catch (e) {
      state = AuthState(

```

```

        isLoading: false,
        errorMessage: e.toString().contains('400')
            ? 'Invalid email or password'
            : 'Connection error',
    );
}
}

```

2.4. Lokalizacja i zastosowanie serwisów mapowych

W aplikacji wyświetlana jest mapa OSM z warstwą rowerową, na nią nakładana jest lokalizacja z GPS.

```

OSMFlutter(
  controller: controller,
  onMapIsReady: (isReady) async {
    if (!isReady) return;

    await controller.drawRoadManually(
      widget.selectedRoute.points,
      const RoadOption(
        roadColor: Colors.blue,
        roadWidth: 12,
      ),
    );

    await controller.moveTo(widget.selectedRoute.points.first);

    if (!_isTracking) {
      _isTracking = true;
      await _startLocationTracking();
    }

    await controller.currentLocation();
  },
  osmOption: const OSMOption(
    userTrackingOption: UserTrackingOption(
      enableTracking: true,
      unFollowUser: false,
    ),
    showDefaultInfoWindow: false,
    showContributorBadgeForOSM: false,
    zoomOption: ZoomOption(
      initZoom: 18,
      minZoomLevel: 12,
      maxZoomLevel: 19,
    ),
    roadConfiguration: RoadOption(
      roadColor: Colors.transparent,
    ),
  ),
),

```

2.5. Serwisy zewnętrzne

Do wyznaczania tras wykorzystano Open Source Routing Machine (OSRM) uruchomiony na serwerze w kontenerze Docker.

Aplikacja komunikuje się z nim za pomocą interfejsu HTTP API. Użytkownik podaje adresy początkowe i końcowe (ewentualnie pośrednie), które następnie są zamieniane na współrzędne geograficzne z wykorzystaniem pakietu geocoding. Po otrzymaniu współrzędnych aplikacja wysyła zapytanie do serwera OSRM, który zwraca jedną lub więcej tras

```
class RouteService {
    static Future<List<RouteOption>> fetchRoute({
        required List<String> waypoints,
        required City city,
    }) async {
        final geoPoints = await geocodeWaypoints(waypoints, city);

        final coords = geoPoints.map((p) => '${p.lon},${p.lat}').join(';');
        final url = 'http://192.168.1.102:5000/route/v1/bike/$coords'
            '?steps=true&overview=full&geometries=geojson'
            '&alternatives=true';

        final response = await http.get(Uri.parse(url));
        final data = jsonDecode(response.body);
        final routes = data['routes'] as List;

        final options = routes.asMap().entries.map((entry) {
            final index = entry.key;
            final route = entry.value;

            final coordinates = route['geometry']['coordinates'] as List;
            final points = coordinates.map<GeoPoint>((c) {
                return GeoPoint(latitude: c[1], longitude: c[0]);
            }).toList();

            final double distance = (route['distance'] as num).toDouble();
            final double duration = (route['duration'] as num).toDouble();

            return RouteOption(
                name: index == 0 ? 'Fast Route' : 'Alternative $index',
                points: points,
                distance: distance,
                duration: duration,
            );
        }).toList();

        return options;
    }
}
```

2.6. Asynchroniczność i komunikacja sieciowa

Funcje sieciowe aplikacji są zaimplementowane asynchronicznie co widać np. w punktach Baza danych 2.1 oraz Serwisy zewnętrzne 2.5.